

Docker — Detailed Notes

1. What is Docker?

Docker is a platform used to package, distribute, and run software inside isolated environments called containers.

In simple words:

Docker puts your program and the things it needs into a container so that it can run consistently on different computers.

For example, suppose you create a ROS 2 robot application.

Your application may need:

- Ubuntu
- ROS 2 Jazzy
- Python
- OpenCV
- Gazebo
- ROS 2 packages
- Other libraries
- Your robot code

Installing all these manually on another computer can be difficult.

Docker helps you create a **container** containing the required software environment.

2. The Problem Docker Solves

Imagine you developed a program on your computer.

Your computer:

- Ubuntu 24.04
- ROS 2 Jazzy
- Python 3.x
- OpenCV version A

Library version A

Your friend's computer:

Ubuntu 22.04

ROS 2 Humble

Python 3.x

OpenCV version B

Library version B

Your program might work perfectly on your computer but fail on your friend's computer.

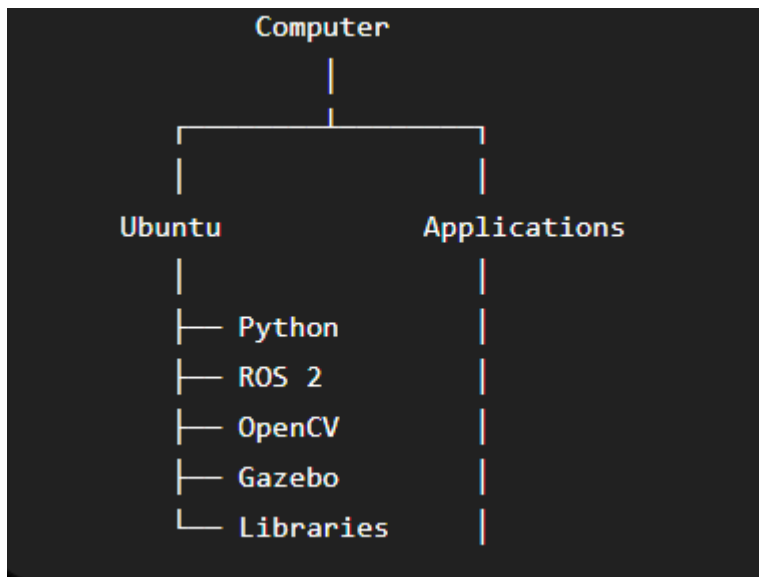
This creates the famous problem:

"It works on my machine!"

Docker helps solve this problem.

3. Before Docker

Without Docker, we normally install software directly onto the computer.



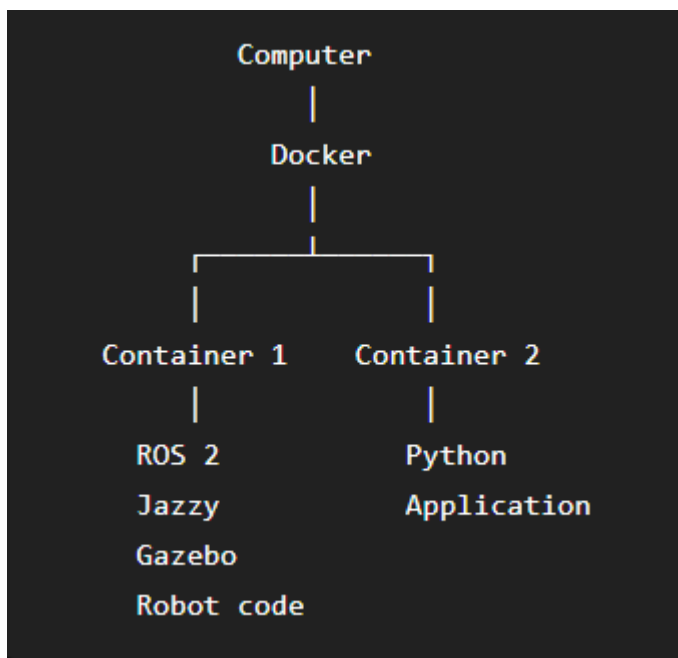
Everything is installed directly into the computer's operating system.

This can create problems:

- Different software versions
 - Dependency conflicts
 - Difficult installation
 - Difficult removal
 - Different environments between computers
-

4. With Docker

Docker creates an isolated environment called a **container**.

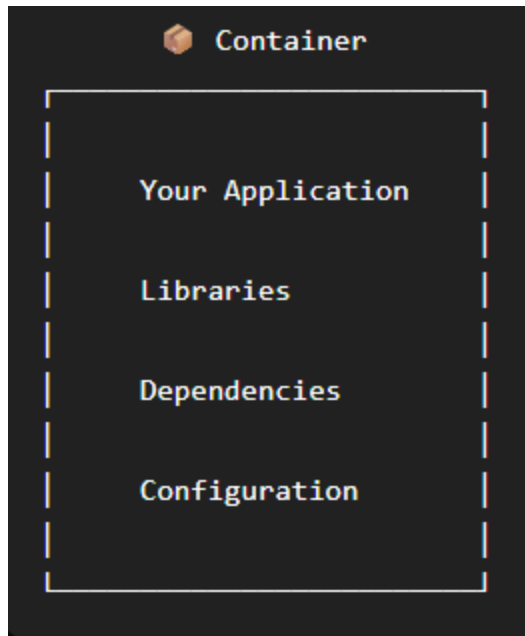


Each container can have its own software environment.

5. What is a Container?

A **container** is an isolated environment in which an application runs.

Think of it like a **box**.



The container provides the environment required by your application.

Simple analogy

Think about a shipping container.

A shipping container can contain:

 Container

Robot
Motors
Sensors
Tools
Batteries
Documents

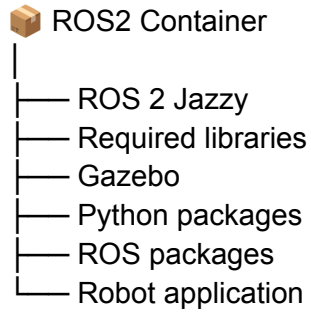
The entire package can be moved from one place to another.

Docker containers work similarly for software.

6. Why is it called a Container?

Because it **contains** everything required to run an application.

For example:



7. What is a Docker Image?

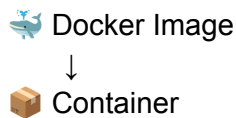
This is one of the most important concepts.

A **Docker image** is a **blueprint/template** used to create containers.

Think of it like a **recipe**.

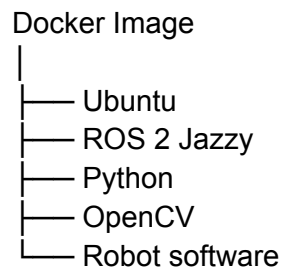


Similarly:



The image describes what should exist inside the container.

For example:



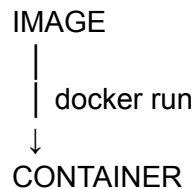
When you run the image, Docker creates a container from it.

8. Image vs Container

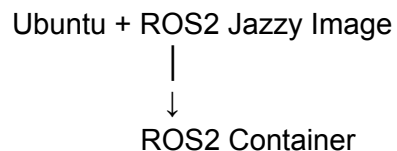
This distinction is extremely important.

Docker Image	Docker Container
Blueprint	Running instance
Template	Actual environment
Used to create containers	Created from an image
Doesn't normally change while running	Can have changes while running
Like a recipe	Like the prepared food

Think:



For example:



9. What is Dockerfile?

A **Dockerfile** is a text file containing instructions for creating a Docker image.

For example:

FROM ubuntu:24.04

RUN apt update

RUN apt install -y python3

CMD ["python3"]

This tells Docker:

Start with Ubuntu 24.04



Update packages



Install Python



Start Python

So:

Dockerfile



Docker Image



Docker Container

This is one of the most important Docker workflows.

10. The Three Main Concepts

Remember these three:

Dockerfile



Docker Image



Docker Container

Dockerfile

Instructions.

Image

Blueprint.

Container

Running environment.

11. What is Docker Engine?

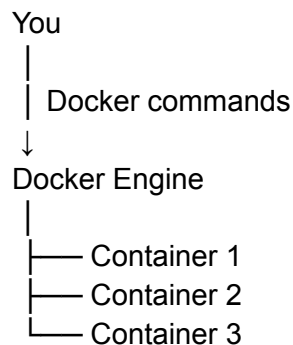
Docker Engine is the software that actually creates and runs containers.

You interact with Docker through commands such as:

```
docker run
docker build
docker ps
docker stop
```

Docker Engine manages the containers.

Conceptually:

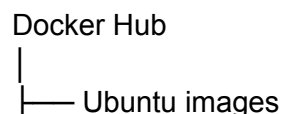


12. What is Docker Hub?

Docker Hub is an online registry where Docker images can be stored and shared.

Think of it like an app store for Docker images.

For example:

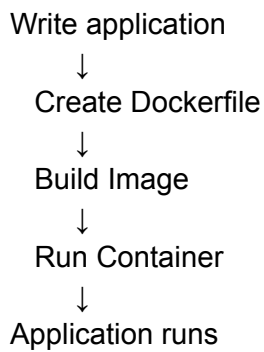


- Python images
- ROS images
- PostgreSQL images
- Many other images

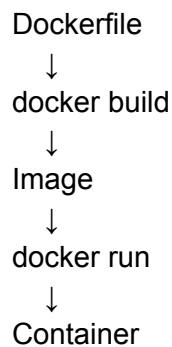
You can download an image and create a container from it.

13. Basic Docker Workflow

The general workflow is:



Or:



14. Installing Docker on Ubuntu

On Ubuntu, Docker can be installed using Docker's official installation instructions.

Install Docker on Ubuntu

Step 1: Update packages

```
sudo apt update
```

Updates Ubuntu's package information.

Step 2: Install Docker

```
sudo apt install docker.io -y
```

Installs Docker.

Step 3: Start Docker

```
sudo systemctl start docker
```

Starts the Docker service.

Step 4: Enable Docker at startup

```
sudo systemctl enable docker
```

Automatically starts Docker when Ubuntu boots.

Step 5: Check installation

```
docker --version
```

Displays the installed Docker version.

Step 6: Check Docker status

```
sudo systemctl status docker
```

Look for:

Active: active (running)

Press **q** to exit.

Step 7: Test Docker

```
sudo docker run hello-world
```

If you see **"Hello from Docker!"**, Docker is working. 

Optional: Use Docker without **sudo**

```
sudo usermod -aG docker $USER
```

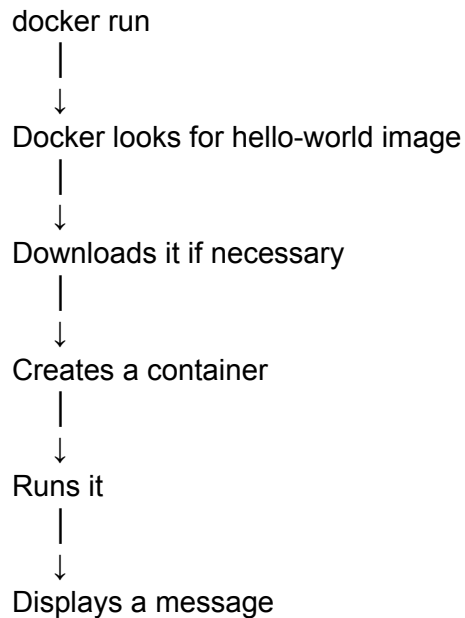
Then **log out and log in again**.

15. Your First Docker Container

A very simple example:

```
docker run hello-world
```

What happens?



This is a good first Docker experiment.

16. Understanding `docker run`

Consider:

`docker run hello-world`

Break it down:

`docker`

Use Docker.

`run`

Create and run a container.

`hello-world`

Use the `hello-world` image.

Therefore:

`docker run hello-world` means "create and run a container using the hello-world image."

17. Running Ubuntu in Docker

You can run an Ubuntu container:

`docker run -it ubuntu`

Here:

`docker`

Docker command

`run`

Run a container

`-it`

Interactive terminal

ubuntu

Ubuntu image

You may see something like:

```
root@container_id:/#
```

Now you are inside the Ubuntu container.

You can try:

ls

or:

apt update

18. Container is NOT a Virtual Machine

This is an important point.

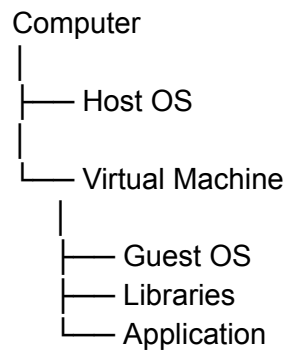
Many beginners think:

"Docker is just a virtual machine."

It isn't.

Virtual Machine

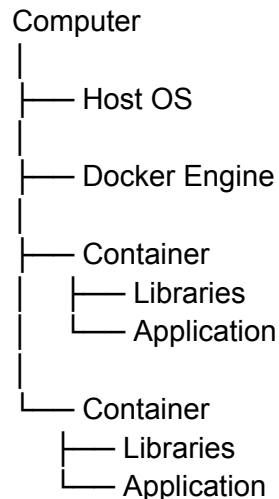
A VM normally looks like:



Each VM has its own operating system kernel.

19. Docker Container

Docker normally works more like:



Containers share the host system's **kernel**, while providing isolated user-space environments.

That's one reason containers are generally lighter than full VMs.

20. VM vs Docker

Virtual Machine	Docker Container
Usually includes a complete guest OS	Usually shares host kernel
Larger	Lightweight
Takes more resources	Generally uses fewer resources
Slower startup	Usually starts quickly
Stronger OS-level separation	Application-level isolation

Good for running different OSes

Good for packaging applications

For example:

VM

Computer
└─ Ubuntu VM
 └─ ROS 2

Docker:

Computer
└─ Docker
 └─ ROS 2 Container

21. Why Docker is Useful in Robotics

Docker is particularly useful in robotics because robotics software often has **many dependencies**.

For example:

ROS 2
├─ rclcpp
├─ rclpy
├─ Gazebo
├─ RViz
├─ OpenCV
├─ PCL
├─ Navigation
├─ SLAM
└─ Hardware drivers

Different projects may require different versions.

Docker can isolate those environments.

22. Example: ROS 2 Jazzy

Imagine you have:

Robot Project A

It requires:

Ubuntu 24.04

ROS 2 Jazzy

OpenCV

Gazebo

You could create:

 robot_project_container

Ubuntu 24.04

+

ROS 2 Jazzy

+

Gazebo

+

OpenCV

+

Robot project

Another project could have a different environment:

 old_robot_container

Ubuntu environment

+

ROS 2 Humble

+

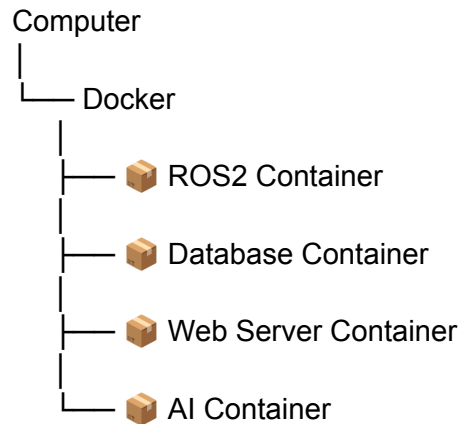
Older dependencies

Both environments can coexist on the same computer without you having to install both ROS distributions directly into the host OS.

23. Multiple Containers

You can run multiple containers.

For example:



Each container can perform a different job.

24. Docker Commands You Should Know

Check Docker version

```
docker --version
```

Show running containers

```
docker ps
```

Example:

```
CONTAINER ID  
IMAGE  
COMMAND  
STATUS
```

Show all containers

```
docker ps -a
```

-a means **all**.

This includes stopped containers.

Download an image

```
docker pull ubuntu
```

This downloads the Ubuntu image.

Run an image

```
docker run ubuntu
```

Run interactively

```
docker run -it ubuntu
```

Stop a container

```
docker stop <container_id>
```

For example:

```
docker stop abc123
```

Start an existing container

```
docker start <container_id>
```

Remove a container

```
docker rm <container_id>
```

List Docker images

```
docker images
```

Remove an image

```
docker rmi <image_id>
```

25. Important Docker Commands Table

Command	Purpose
<code>docker --version</code>	Check Docker version
<code>docker pull</code>	Download image
<code>docker images</code>	List images
<code>docker run</code>	Create and run container
<code>docker ps</code>	Show running containers
<code>docker ps -a</code>	Show all containers
<code>docker start</code>	Start stopped container
<code>docker stop</code>	Stop container
<code>docker restart</code>	Restart container
<code>docker rm</code>	Remove container
<code>docker rmi</code>	Remove image
<code>docker build</code>	Build image
<code>docker logs</code>	View container output
<code>docker exec</code>	Execute command inside container

26. What is `docker exec`?

Suppose you already have a running container.

You can enter it using:

```
docker exec -it <container_id> bash
```

For example:

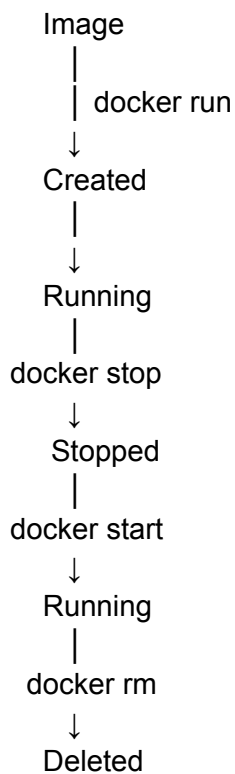
```
docker exec -it abc123 bash
```

You are essentially saying:

"Open a terminal inside this running container."

27. Container Lifecycle

A container has a lifecycle.



Understanding this makes Docker commands much easier.

28. What Happens When You Run **docker run**?

Suppose:

```
docker run ubuntu
```

Docker roughly does this:

1. Check whether Ubuntu image exists
 - ↓
 2. Download image if necessary
 - ↓
 3. Create container
 - ↓
 4. Start container
 - ↓
 5. Run the specified command
-

29. What is a Docker Registry?

A **Docker registry** stores Docker images.

Example:

Docker Hub



You can pull an image:

```
docker pull ubuntu
```

Docker downloads it from a registry.

30. What is a Docker Repository?

A repository is a place where versions of a particular image are stored.

For example, conceptually:

Repository: ubuntu

ubuntu:24.04

ubuntu:22.04

ubuntu:latest

These are different tags/versions of the Ubuntu image.

31. What is a Docker Tag?

A tag identifies a particular version or variant of an image.

For example:

ubuntu:24.04

Here:

ubuntu

↑

image name

24.04

↑

tag

You can run:

`docker run -it ubuntu:24.04`

32. Why Should We Specify Versions?

Suppose you use:

`docker run ubuntu`

The image selected by the default tag may change over time.

For reproducible projects, it is often better to specify the required version:

```
docker run -it ubuntu:24.04
```

This is particularly useful for educational and robotics projects.

33. Dockerfile Example

Let's create a simple Python application.

app.py

```
print("Hello from Docker!")
```

Dockerfile

```
FROM python:3.12
```

```
WORKDIR /app
```

```
COPY app.py .
```

```
CMD ["python3", "app.py"]
```

Now let's understand each line.

34. FROM

```
FROM python:3.12
```

This says:

Start with a Python 3.12 image.

Python Image

↓

Our Docker Image

35. WORKDIR

WORKDIR /app

This creates/sets the working directory:

/app

Commands will operate from there.

36. COPY

COPY app.py .

This copies our Python program into the container.

Computer

app.py
|
| COPY
↓

Container

/app/app.py

37. CMD

CMD ["python3", "app.py"]

This tells Docker:

When the container starts, run `python3 app.py`.

38. Build the Image

Suppose your folder contains:

```
my_project/
|
|— Dockerfile
|— app.py
```

Run:

```
docker build -t my-python-app .
```

Here:

```
docker build
```

Build an image.

```
-t my-python-app
```

Give the image a name.

.

Use the current directory as the build context.

39. Run Your Image

After building:

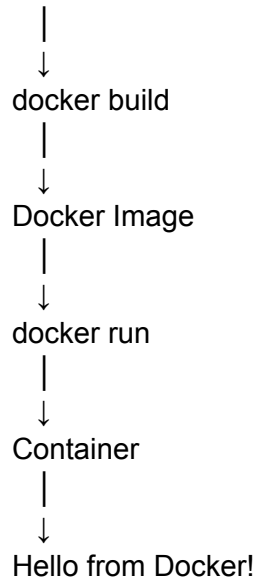
```
docker run my-python-app
```

Output:

Hello from Docker!

The flow is:

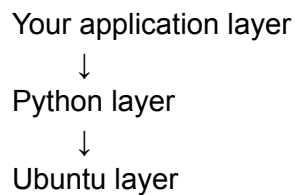
```
app.py
+
Dockerfile
```



40. Docker Image Layers

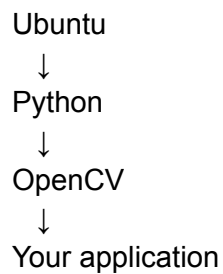
Docker images are generally built in **layers**.

For example:



This allows Docker to reuse layers and avoid rebuilding everything unnecessarily.

For example:



If you only change your application code, Docker may be able to reuse the lower layers.

41. Docker Volumes

Here's an important problem.

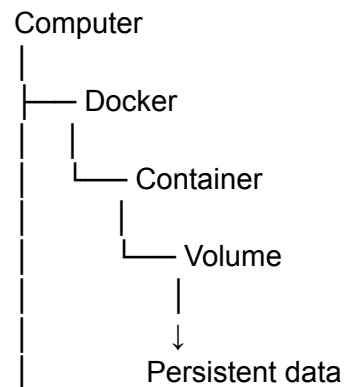
Suppose you create files inside a container.

Then you delete the container.

Those files may disappear along with the container's writable layer.

What if you want data to survive?

Use a **volume**.



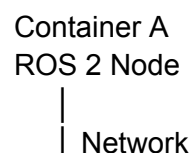
Volumes are commonly used for:

- databases
- project data
- configuration
- persistent application files

42. Docker Networking

Containers can communicate with each other using Docker networking.

For example:



↓
Container B
ROS 2 Node

This is especially interesting for robotics systems.

You could have:

📦 Container 1
Robot drivers

📦 Container 2
Navigation

📦 Container 3
Computer Vision

and allow them to communicate.

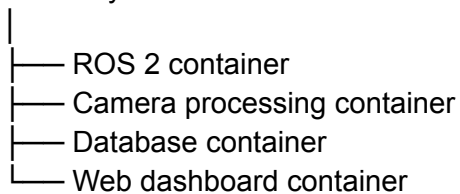
43. Docker Compose

When a project contains multiple containers, managing them individually can become inconvenient.

That's where **Docker Compose** comes in.

For example:

Robot System

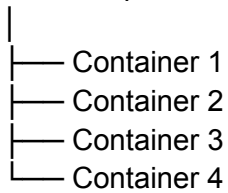


Instead of manually starting everything, you can define the services in a Compose file and start them together.

Conceptually:

compose.yaml
↓

Docker Compose



A common command is:

`docker compose up`

44. Docker in a Robotics Project

Imagine your autonomous rover has:

Camera



Computer Vision



ROS 2

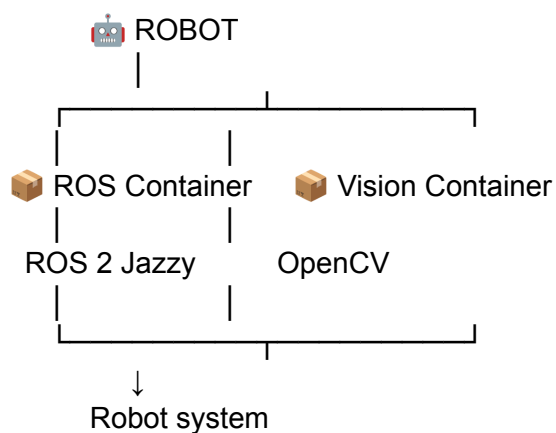


Navigation



Motor Controller

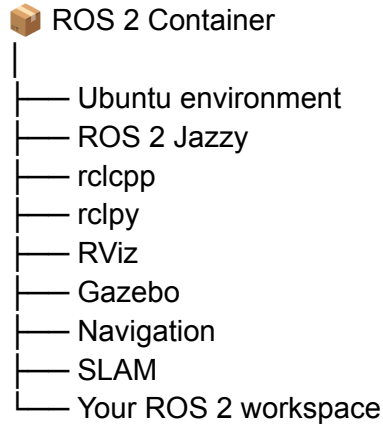
You could organize software like:



Docker can help keep these software environments reproducible.

45. Docker and ROS 2

For ROS 2 development, a typical Docker environment might contain:

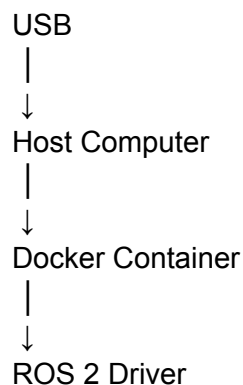


Your host computer provides the underlying hardware/kernel, while the container provides the software environment.

46. Can Docker Access Robot Hardware?

Yes, but it requires configuration.

For example:



You may need to explicitly pass devices into the container.

For example, Linux USB devices can sometimes be passed with options such as:

--device

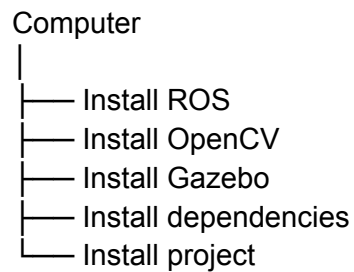
Hardware access becomes more important with:

- Arduino
- ESP32
- LiDAR
- USB cameras
- serial devices
- motor controllers

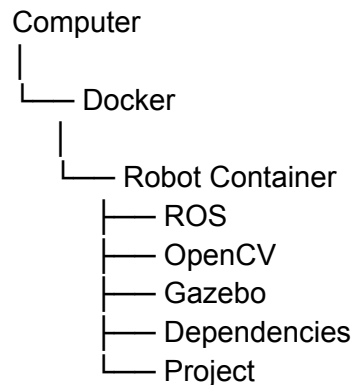
So Docker doesn't automatically give a container access to every hardware device.

47. Docker vs Normal Installation

Normal installation



Docker



Docker makes the software environment easier to reproduce.

48. Advantages of Docker

1. Consistent environment

The same image can be used by different developers.

2. Dependency isolation

Different projects can use different dependencies.

3. Easy deployment

You can package the application environment and deploy it elsewhere.

4. Lightweight

Containers are generally lighter than full virtual machines.

5. Reproducibility

You can recreate the same environment from a Dockerfile/image.

6. Easy cleanup

Instead of uninstalling dozens of packages, you can remove the container.

49. Disadvantages of Docker

Docker is not magic.

Some disadvantages include:

1. Learning curve

There are several concepts:

- Image
- Container
- Dockerfile
- Volume
- Network
- Registry
- Compose

2. Hardware access

Accessing GPUs, USB devices, cameras, serial ports, etc. can require extra configuration.

3. GUI applications

GUI applications such as RViz or Gazebo may need additional configuration depending on the setup.

4. Storage

Docker images can consume significant disk space.

5. Networking complexity

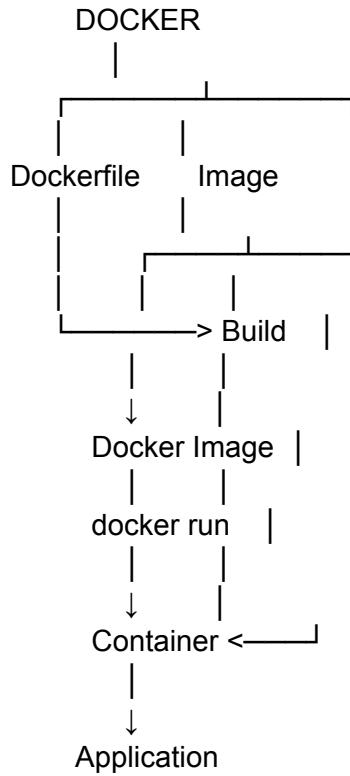
Networking becomes more complicated when many containers need to communicate.

50. Important Terms to Remember

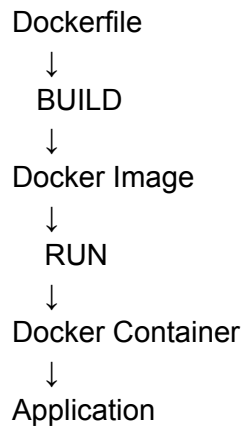
Term	Simple Meaning
Docker	Platform for running containers
Container	Isolated environment running an application
Image	Blueprint used to create a container
Dockerfile	Instructions for building an image
Docker Engine	Runs and manages containers
Docker Hub	Online image registry
Volume	Persistent storage
Network	Allows containers to communicate
Compose	Manages multiple containers
Registry	Stores Docker images

51. The Most Important Diagram

Memorize this:



Or even simpler:



52. Docker Commands — Beginner Practice

You can practice these in order:

Step 1

Check Docker:

```
docker --version
```

Step 2

Run the test container:

```
docker run hello-world
```

Step 3

Download Ubuntu:

```
docker pull ubuntu
```

Step 4

Run Ubuntu:

```
docker run -it ubuntu
```

Step 5

Inside the container:

```
ls
```

Step 6

Exit:

```
exit
```

Step 7

See containers:

`docker ps -a`

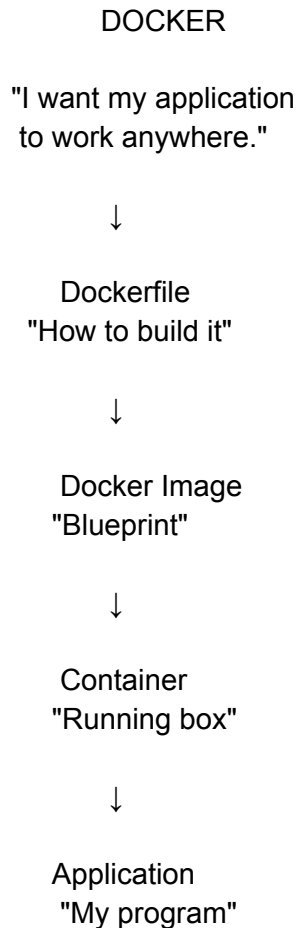
Step 8

See images:

`docker images`

53. A Simple Mental Model

If you remember only this, you're already on the right track:



One-line definitions for exams/interviews

Docker:

A platform for developing, packaging, and running applications in containers.

Container:

An isolated environment in which an application and its dependencies run.

Image:

A read-only template/blueprint used to create containers.

Dockerfile:

A text file containing instructions to build a Docker image.

Docker Hub:

A registry for storing and sharing Docker images.

54. Docker in Your ROS 2 Learning Path

Since you're working with **ROS 2 Jazzy**, I would learn Docker in this order:

1. What is Docker?
↓
2. Image vs Container
↓
3. Dockerfile
↓
4. docker pull
↓
5. docker run
↓
6. docker ps
↓
7. docker exec
↓
8. Volumes
↓
9. Networking
↓
10. Docker Compose
↓
11. ROS 2 + Docker

↓
12. ROS 2 Jazzy + Gazebo + Docker

↓
13. Docker + your actual rover

The most important thing for you now is not memorizing commands. First understand this relationship:

Dockerfile → Image → Container → Application

Once that clicks, the rest of Docker becomes much easier.